Multi-Disease Prediction Using Machine Learning: An Intelligent Health Assistant for Diabetes, Heart, Liver and Kidney Risk

Ajmeera Harini
Roll No. 23K91A0507
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

Bandi Likitha
Roll No. 23K91A0526
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

Chinthakindi Swathi
Roll No. 23K91A0555
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

Badroju Sri Hansika
Roll No. 23K91A0524
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

B. Sruthi
[Designation], Project Guide
Dept. of Computer Science and Engineering
TKR College of Engineering and Technology (Autonomous)
[City]
[email]

***Abstract*---**Chronic diseases are usually screened one at a time, and published systems rarely cover several conditions from routine laboratory values. This paper presents an Intelligent Health Assistant that screens diabetes, heart disease, liver disease and kidney disease through one web interface. For every condition, Logistic Regression, a Decision Tree, a Random Forest and an RBF Support Vector Machine are trained on standardised inputs, and the model with the highest F1-score on a stratified 80/20 hold-out is served by a FastAPI service and a React application. The four training sets are synthetic: 2,000 rows each, labelled by a linear risk rule with 8% of the labels flipped. The served models therefore reach accuracies of 89.5%, 88.5%, 91.8% and 88.3%, close to the 92% ceiling that the noise allows, and these numbers are not clinical results. Cross-validation shows Logistic Regression to be the most reliable model for every condition. On real public data the same algorithms score 72% to 99%, and the served models transfer only partly. The application gives no explanations and is a demonstration, not a diagnostic tool.

***Index Terms*---**diabetes prediction, healthcare analytics, logistic regression, machine learning, multi-disease prediction, risk assessment, support vector machine.

I. INTRODUCTION

Diabetes, heart disease, liver disease and kidney disease are common chronic conditions whose early stages are often silent. Routine laboratory values and simple measurements such as glucose, blood pressure, cholesterol, bilirubin, creatinine and haemoglobin already carry early warning signals, and supervised learning can turn them into a risk estimate [2], [3]. Today a patient who wants these checks usually visits several specialists and takes a separate set of tests for each, and in rural centres there may be nobody to interpret the report. Late diagnosis then raises both treatment cost and mortality.

Published machine-learning systems mostly address one disease at a time: diabetes on the Pima data [4], kidney disease [3], liver disease [5]. Multi-disease work tends to use reported symptoms instead of laboratory values [6], [7], and the one multi-disease web application we found covers stroke, depression and diabetes [8]. The base paper of this work studies a single condition, thyroid disease, with rigorous validation and explanations [1]. The gap is one pipeline that screens several conditions from routine clinical values, compares the same classical algorithms on each, and reports the evidence honestly.

The contributions of this paper are: (1) a web application that serves risk predictions for four conditions behind one interface; (2) a like-for-like comparison of four classical algorithms on each condition, selected by F1-score; (3) an audit of that comparison with ROC-AUC, cross-validation, ten random splits, threshold and sensitivity analysis, which changes one of the four model choices; and (4) a check on real public data that shows how far the synthetic-data results can be trusted. Section II reviews the literature, Section III gives the method, Section IV the results and Section V concludes.

A. System Hardware

All training and measurements ran on one Windows 11 desktop with an Intel Core i5-12500 processor (12 threads) and 16 GB of RAM. No GPU was used; training the 16 models takes a few seconds on the CPU.

B. Software Requirements

Python 3.11.9 with scikit-learn 1.8.0 [9], pandas 3.0.2, NumPy 2.4.4 and joblib 1.5.3 for training; Flask 3.1.3 for the original pages; FastAPI 0.110.0 with Uvicorn 0.29.0 for the API; React 19 with Vite 8 on Node 24 for the interface; Playwright with Chrome for the screen captures.

II. LITERATURE SURVEY

Base paper. Alam et al. [1] classify thyroid status (normal, hypothyroid, hyperthyroid) on the UCI thyroid data, 7,200 samples with 21 features. They train five classifiers (SVC, Random Forest, Extra Trees, AdaBoost, CatBoost) under nested cross-validation, with an outer stratified 5-fold loop for assessment and an inner 3-fold grid search for tuning, on the full feature set and on a ten-feature consensus subset. Random Forest and CatBoost reach 0.996 to 0.997

accuracy and an AUC of 0.999 to 1.000, and SVC 0.965 to 0.967. They add SHAP attributions and counterfactual what-if explanations, test SMOTE and reject it because minority recall rose by under 2%, and name class imbalance and the lack of external validation as limits. Our work borrows the idea of comparing several classical classifiers under held-out evaluation, but covers four diseases, uses a single split with fixed settings, and does not explain individual predictions.

Single-disease systems. On the Pima diabetes data, six classifiers were compared and gradient boosting was best at 81.8% accuracy, with the warning that results may hold only for that sub-population [4]; the data set stems from a 1988 onset-forecasting study [10]. For kidney disease, twelve classifiers gave XGBoost 0.983 accuracy [3], and an earlier study on 400 individuals reported an F1-measure of 0.993 while ranking the predictive attributes [2]. For liver disease, re-created models on the Indian Liver Patient Dataset reached about 71% to 79% accuracy and made more false-negative errors for women [5].

Multi-disease systems. A Bangla symptoms-disease set of 758 relations over 85 diseases gave 0.98 accuracy with voting ensembles and 0.97 with Random Forest [6]; a survey-based study predicted 11 common diseases from symptoms [7]. A 2025 Django application bundles separate models for stroke, student depression and diabetes and lowers the diabetes threshold to 0.2 to raise recall [8]. Selecting a model on the data used to report its error is optimistic [11], which is why the base paper nests its tuning; this project selected on one hold-out and carries that bias.

Explanations, imbalance and data. SHAP assigns each feature an additive contribution to one prediction [12], and a counterfactual explanation states the smallest change of the inputs that would alter a decision [13]; the base paper uses both and notes their computational overhead [1]. Oversampling with SMOTE [14] was tried there and rejected. Four public benchmarks match our four conditions: Pima diabetes [10] (768 patients, 8 features, physiologically impossible zeros that act as missing values), UCI Cleveland heart disease [15] (303 instances, 13 features), the Indian Liver Patient Dataset [16] (583 instances, 416 of them patients, 10 features) and UCI chronic kidney disease [17] (400 instances, 250 with the disease, 24 features, with missing values). They are small and incomplete, and published accuracies vary with preprocessing and split, from 81.8% on Pima [4] to 0.983 on kidney data [3].

Reading the base paper against this project. Its strengths are real data, an unbiased protocol in which tuning never sees the assessment folds, class-wise reporting under imbalance and explanations that agree with endocrine physiology [1]. Its limits, which it states itself, are one disease, a skewed class distribution and no external validation. For a multi-disease screen the transferable lessons are the evaluation discipline and the explanation layer; this project adopts the first only in part (cross-validation and repeated splits are run afterwards, not used for selection) and not the second.

TABLE I
SUMMARY OF THE REVIEWED WORK

Work	Approach	Data	Reported result	Limitation
Alam et al. [1]	5 classifiers, nested CV, SHAP	UCI thyroid, 7,200	Acc. 0.996-0.997	One disease
Zannat et al. [6]	8 classifiers, voting	Bangla symptoms, 85 diseases	Acc. 0.98 (ensemble)	Symptoms only
Kamakshamma et al. [8]	One model per disease, web app	Kaggle: stroke, depression, diabetes	Recall 94.73, 88.39, 91.02%	Other diseases
Sun and Sun [4]	6 classifiers	Pima diabetes	Boosting 81.8%	One sub-population
Islam et al. [3]	12 classifiers, feature reduction	CKD, 25 variables	XGBoost 0.983	One disease
Salekin, Stankovic [2]	Attribute ranking, classifier	400 people, 250 CKD	F1 0.993	Small cohort
Straw, Wu [5]	Re-created liver models	ILPD	Acc. 71-79%	Bias, no fix

None of these combines diabetes, heart, liver and kidney risk from routine clinical values behind one interface and then audits its own model selection; that is the gap this work addresses.

III. METHODOLOGY

A. Data

No clinical records were available when the system was built, so generate_data.py creates four data sets of 2,000 rows each with a fixed seed (42): diabetes (8 features), heart disease (13), liver disease (10) and kidney disease (12), 8,000 rows in all. Features are drawn from clipped normal, exponential and uniform distributions with medically informed means. A hand-written linear risk score (for example 0.04 (glucose - 120) + 0.06 (BMI - 30) + 0.02 (age - 40) + 1.5 DPF + 0.03 (BP - 72) - 1.4 for diabetes) is passed through a sigmoid, thresholded at 0.5, and then **8% of the labels are flipped at random**. The label is therefore a known linear function of the inputs plus noise, and no classifier can exceed an expected accuracy of 1 - 0.08 = 92% on fresh draws. Positive rates are 41%, 73%, 28%, 60% for the four sets, with no missing values and no duplicates. Each set is split 80/20 with stratification (1,600 training rows, 400 test rows, seed 42). The heart rule weights age, sex, chest-pain type, blood pressure, cholesterol, exercise angina, ST depression and the number of vessels, and (negatively) maximum heart rate; the liver rule weights bilirubin, alkaline phosphatase, ALT, AST and age, and (negatively) albumin; the kidney rule weights age, blood pressure, albumin, sugar, glucose, urea and creatinine, and (negatively) specific gravity and haemoglobin. Other columns, such as insulin and skin thickness for diabetes, fasting sugar, ECG and slope for heart disease, or sodium and potassium for kidney disease, carry no weight and are noise by construction.

B. Preprocessing and models

The only preprocessing is a StandardScaler fitted on the training part; there is no imputation, outlier handling or feature selection, because the synthetic data needs none. Four algorithms are trained per disease with fixed settings and no tuning: Logistic Regression (maximum 1,000 iterations), a

Decision Tree (depth 6), a Random Forest [18] of 200 trees (depth 8) and an RBF Support Vector Machine [19] (C = 1, probability outputs). Logistic Regression models p(y = 1 | x) = 1 / (1 + exp(-(w.x + b))) and minimises the log-loss; the SVM maximises a margin in the RBF feature space; the forest averages decorrelated trees.

C. Selection and evaluation

The model with the highest F1-score = 2PR / (P + R) on the 400-row test set is saved with its scaler and served; accuracy, precision and recall are recorded as well. The training script does not compute ROC-AUC, so we measured it afterwards, together with 5-fold cross-validation (scaler refitted inside each fold), ten different 80/20 splits, a decision-threshold sweep, permutation importance and hyper-parameter sensitivity. Permutation importance shuffles one feature of the test set 20 times and records the drop in accuracy of the served model; the threshold sweep moves the cut-off on the positive-class probability from 0.20 to 0.80 in steps of 0.05; the sensitivity runs re-train trees at depths 2 to 12, forests with 10 to 400 trees and the SVM with C from 0.1 to 100. These re-runs reproduce the served report exactly (all 16 model-disease cells match to four decimals).

D. Architecture and implementation

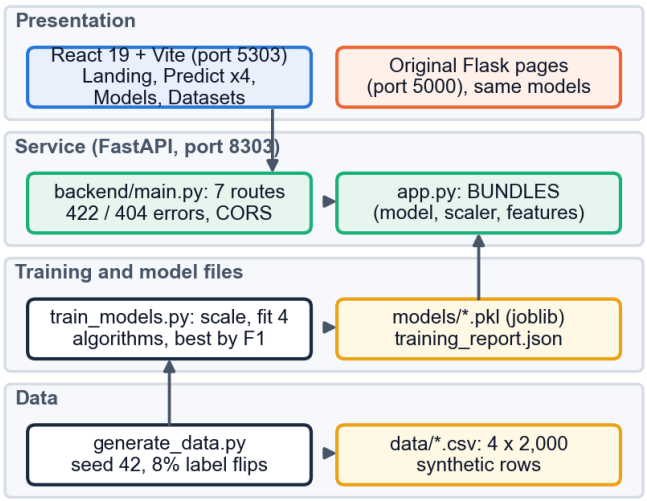


Fig. 1. System architecture: synthetic data and training scripts at the bottom, model files, the FastAPI service that imports the Flask module and the React interface.

app.py loads each disease's (model, scaler, feature list) bundle and the field metadata; backend/main.py imports it, so the API and the original Flask pages give identical predictions. Seven routes exist: health, disease list, disease detail, a real sample row, prediction, report and dataset summary. The React interface has a form for each disease with buttons that fill it from a real data row, a model-comparison page and a dataset page that warns that the data are synthetic. Nothing is stored: there is no database, no user account and no log of predictions.

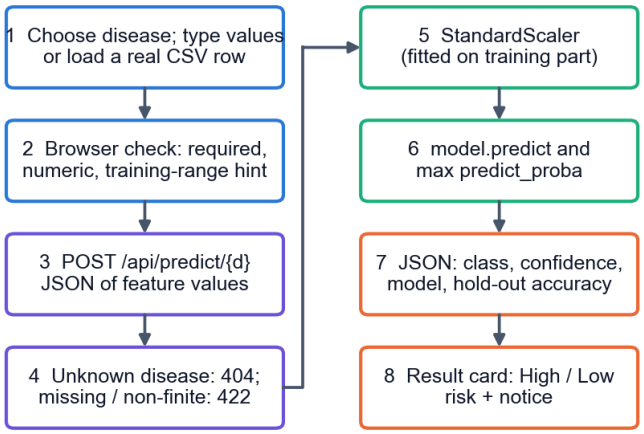


Fig. 2. Main flow of one prediction, from the form to the result card.

A request carrying every feature value is converted to numbers; a missing or non-finite value returns HTTP 422 with a message and an unknown disease returns 404. Otherwise the values are scaled, the model predicts the class, and the reported confidence is the larger class probability, which lies between 50% and 100%. It is a measure of how sure the model is of its own answer, not the probability of disease.

Four design decisions follow from the setting. The API imports the Flask module instead of copying it, so a model is trained and loaded in one place and the two front ends cannot disagree. The scaler is saved next to each model because a model is only valid on inputs scaled the same way. F1 rather than accuracy selects the model because the positive class is a minority in three of the four sets. The interface shows the confidence next to the model's hold-out accuracy so that a reader sees how much the figure is worth.

E. Reproducibility

Every number in this paper is produced by a script in the repository: generate_data.py (seed 42), train_models.py, run_experiments.py (ROC-AUC, confusion matrices, cross-validation, ten splits, thresholds, importance, sensitivity, timings), real_data_check.py (the real-data check), api_tests.py (the API checks and latency) and capture scripts that drive the real interface in Chrome to produce the screens. The results are written as JSON files and the documents read those files, so the text cannot drift from the measurements. The original build had no access to public data, and the Datasets page of the application says that its data are synthetic.

F. Supplementary check on real data

To test how far the synthetic results transfer, the same four algorithms were cross-validated (5 folds repeated 3 times, median imputation and scaling inside each fold) on Pima diabetes [10] (768 rows), UCI Cleveland heart disease [15] (303), the Indian Liver Patient Dataset [16] (583) and UCI chronic kidney disease [17] (400, the 12 columns that match our schema), and the served models were applied to

every real record after mapping the codes to ours. This check is not part of the application.

IV. RESULTS

All numbers come from the 400-row test set of each disease (seed 42) unless stated. TABLE II lists the model that the application serves for each disease; the ranking of the other algorithms is shown in the repository's model-comparison page and in the report.

TABLE II
HOLD-OUT METRICS OF THE SERVED MODEL FOR EACH DISEASE (SYNTHETIC DATA, 400 TEST ROWS)

Disease	Model	Acc.	Prec.	Rec.	F1	AUC
Diabetes	SVM	0.8950	0.9110	0.8210	0.8636	0.9077
Heart disease	LR	0.8850	0.8917	0.9589	0.9241	0.8455
Liver disease	LR	0.9175	0.9158	0.7768	0.8406	0.8881
Kidney disease	LR	0.8825	0.8934	0.9121	0.9027	0.9076

Accuracy of 88.3% to 91.8% sits just under the 92% noise ceiling, so the pipeline recovers the generating rule almost as well as is possible; it says nothing yet about patients. Logistic Regression is the served model for heart, liver and kidney disease and SVM for diabetes. The diabetes choice is fragile: SVM beats Logistic Regression by only 0.0038 in F1, whereas 5-fold cross-validation gives accuracy 0.8955 for Logistic Regression and 0.8765 for SVM, and across ten random splits Logistic Regression has the best F1 in 10 of 10 splits for all four diseases. Cross-validated Logistic Regression accuracy is 0.8955, 0.9040, 0.9035 and 0.8780. The Random Forest fits the training part to 93.6%-95.9% but scores 81.5%-88.3% on the test part: with 8% random label noise a deeper model memorises noise. The linear model matches the linear generating rule, so this ranking is a property of the data and should not be taken to the clinic.

Across algorithms the Decision Tree has the lowest ROC-AUC for every disease (0.7339 to 0.8072), while Logistic Regression, Random Forest and SVM lie within 0.027 of each other for each disease. In 5-fold cross-validation the Decision Tree is also the weakest on accuracy (0.7160 to 0.8650) and its depth-6 setting over-fits the label noise; Logistic Regression leads the Random Forest by 0.025 to 0.074 and the SVM by 0.018 to 0.026 in cross-validated accuracy, margins of about one to several fold-to-fold standard deviations (at most 0.017), so the linear model is ahead on this data, clearly over the forest and narrowly over the SVM.

The error profile differs by disease. Confusion counts of the served models (true negatives, false positives, false negatives, true positives) are diabetes 225, 13, 29, 133; heart disease 74, 34, 12, 280; liver disease 280, 8, 25, 87; kidney disease 135, 26, 21, 218. The heart model over-calls (34 false positives against 12 false negatives), as 73% of its rows are positive; the liver model under-calls (25 false negatives against 8 false positives) with 28% positives. Tuning would change little: trees peak at depth 4 to 8 and degrade beyond it, forest accuracy is flat above 50 trees, and the SVM is best at C of 0.3 to 1 and drops to 77% to 83% at C = 100; the fixed settings lie on the plateau, but no grid search was run.

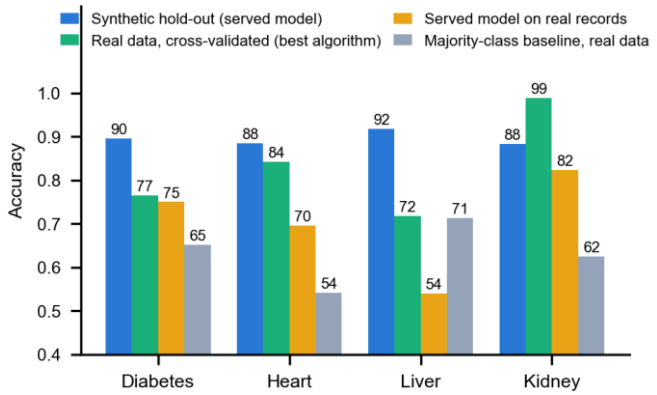


Fig. 3. Accuracy of the served model on the synthetic hold-out, of the best algorithm cross-validated on real public data, of the served model on the real records, and the majority-class rate.

Threshold and drivers. The application uses the 0.5 rule. A sweep from 0.20 to 0.80 changes little: the best F1 is at 0.5 for diabetes, liver and kidney disease and at 0.55 for heart disease (0.9257 against 0.9241). Recall of 0.90 is reached for diabetes only at threshold 0.20 and a precision of 0.6562, and never for liver disease, whose recall is 0.7768. Permutation importance ranks Glucose first for diabetes (0.313 accuracy drop), ca for heart disease (0.108), Alkaline_Phosphotase for liver disease (0.280) and albumin for kidney disease (0.151); these are the inputs that carry weight in the generating rule. This analysis was run offline: the application does not show it.

Real data. On the public data the best of the four algorithms scores 76.5% (diabetes, Logistic Regression), 84.3% (heart, Logistic Regression), 71.8% (liver, Logistic Regression) and 98.8% (kidney, Random Forest), below the synthetic figures, markedly so for diabetes and liver disease. The served models, applied to the real records, reach accuracies of 75.0%, 69.6%, 54.0% and 82.3% with ROC-AUC 0.80, 0.88, 0.75 and 0.99. The ranking of risk therefore carries over, but the 0.5 rule does not: for liver disease the accuracy of 54.0% is below the 71.4% obtained by always predicting the majority class. The kidney set is almost separable, and these real-data numbers are the better guide to how the method behaves on patients.

TABLE III
REAL PUBLIC DATA: CROSS-VALIDATED BEST ALGORITHM AND THE SERVED MODEL APPLIED TO THE REAL RECORDS

Disease	Rows	Best CV acc.	Served acc.	Served AUC	Majority
Diabetes	768	76.5%	75.0%	0.796	65.1%
Heart disease	303	84.3%	69.6%	0.884	54.1%
Liver disease	583	71.8%	54.0%	0.745	71.4%
Kidney disease	400	98.8%	82.3%	0.992	62.5%

Comparison with the base paper. The base paper reports 0.996 to 0.997 accuracy on 7,200 real thyroid records with nested cross-validation and SHAP [1]. Our figures are not comparable: the disease, the data (real versus synthetic), the number of classes (three versus two), the protocol (nested cross-validation with tuning versus one split with fixed

settings) and the explanation layer all differ. Our real-data cross-validated accuracies of 72% to 99% are the closest analogue, and they show how much lower results on noisy real clinical data can be than on a clean benchmark.

Output screens. Fig. 4 and Fig. 5 are captures of the running application. Each prediction page offers a button that fills the form from a real row of the training file, shows beside every field the training range of that feature, warns when a typed value falls outside it (a glucose of 900, for example), marks every empty required field when the form is submitted, and shows a result card with the class, the confidence bar, the model used and its hold-out accuracy together with a notice that the estimate is not a diagnosis. The layout also works on a 390-pixel phone screen. The comparison page lets a reader sort the four algorithms by any metric.

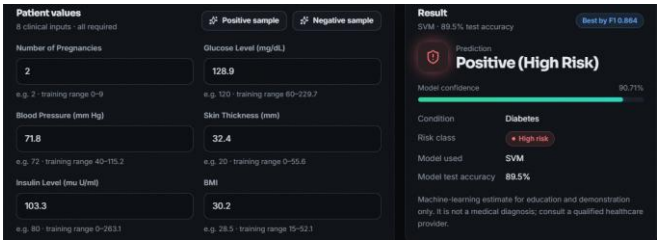


Fig. 4. Output screen: diabetes prediction for a real data row, showing the inputs with their training ranges and the result card (High Risk, 90.71% confidence, SVM, 89.5% hold-out accuracy).

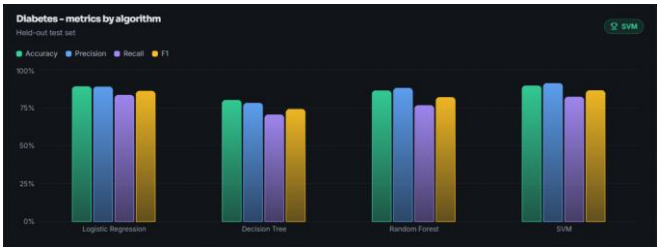


Fig. 5. Output screen: model-comparison page, with accuracy, precision, recall and F1 of the four algorithms for diabetes and the selected model marked.

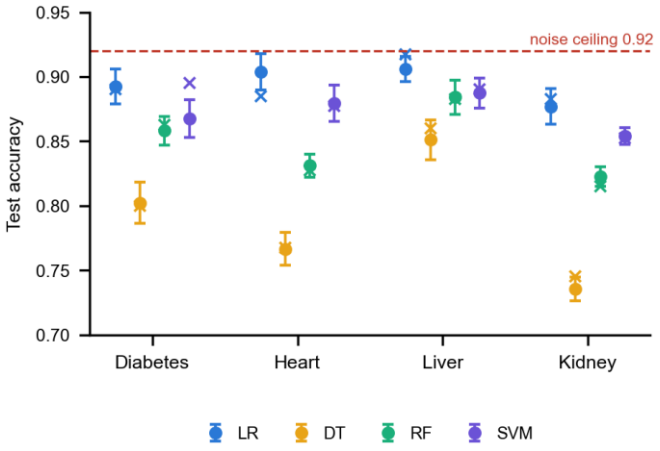


Fig. 6. Mean test accuracy of the four algorithms over ten random 80/20 splits (dots with one standard deviation) and on the seed-42 split (crosses); the dashed line is the 92% ceiling.

Cost. Fitting is cheap: Logistic Regression takes 3 to 12 ms per disease, the SVM 0.15 to 0.27 s and the Random Forest 0.30 to 0.62 s on the CPU described in Section I-A. Scoring one row takes under 0.06 ms in process, so the model is never the bottleneck of a request; the linear model is also the cheapest to store and the easiest to audit.

Threats to validity. The models were selected on the same 400 rows that report their accuracy, which biases the figures upward [11]; the hold-out is small (one point of accuracy is four rows); the synthetic labels follow a linear rule, which favours linear models; the reported confidence is not calibrated; and nothing was validated externally or with clinicians. The ten-split and cross-validation figures in Fig. 6 reduce the first problem but not the others.

Failure cases and tests. For a heart record labelled low risk, the model returned High Risk with 99.53% confidence, a false positive that the confidence figure does not warn about. The API accepts impossible values such as a glucose of 100,000 mg/dL (answered High Risk, 60.94%) because only the interface warns about values outside the training range, and nothing is imputed. The 41 API checks (functional, validation, consistency with direct model calls, sanity) passed, as did a 27-check smoke test; there is no unit-test suite, no user study and no clinical validation. Median latency of a prediction request on the local machine was 13.5 ms (95th percentile 24.9 ms), and 416 requests per second with four concurrent clients, measured while the PC was busy and with coarse timer resolution, so they are upper bounds.

V. CONCLUSION

The Intelligent Health Assistant delivers what the objectives ask for in its core: four algorithms were trained and compared for each of four diseases and one web interface serves the selected models. Several objectives are met only in part: the data are synthetic rather than benchmark data, no missing-value or outlier handling exists, ROC-AUC was not part of the selection, and the application does not show which parameters drove a prediction. The key finding is that on this data a plain Logistic Regression is the most reliable model for every disease, that the shipped diabetes choice rests on a difference inside the noise, and that accuracy near 90% on synthetic data falls to 72% to 84% on three of the four real sets and, for the served liver model, below the majority-class baseline.

Measured against the six stated objectives, two are achieved (training and comparing the four algorithms for each disease, and serving the selected models behind one interface) and four only in part (benchmark data, preprocessing, selection by ROC-AUC, and showing the parameters that drove each prediction). The abstract's promised health recommendations are not implemented. The system is therefore best described as a working demonstration of a multi-disease screening pipeline together with an honest audit of its own evidence.

Limitations are the synthetic data, model selection on the test set, uncalibrated confidence values, no range validation and no explanations. Future work: train on the real benchmark data with imputation and nested cross-validation

as in the base paper [1]; add SHAP [12] and counterfactual [13] explanations and show the driving parameters; calibrate the probabilities and tune thresholds per disease, testing SMOTE [14] only if it helps; add range validation, authentication and an audit log; and validate with clinicians before any real use.

Responsible use. The application stores no patient data, shows a not-a-diagnosis notice on every result and states on its dataset page that the training data are synthetic. Even with real data, a screening aid of this kind should be introduced with clinical oversight, a stated intended use and monitoring of its errors for different groups of patients, since the liver study cited above found a higher false-negative rate in women [5].

REFERENCES

[1] M. Z. Alam, R. Rahman, H. M. Sozib, H. Ahmed, A. Hossain, A. A. Sabeena, A. F. Tasnim, F. Ahmed, M. I. Sarkar, and T. I. Erdei, "Enhancing thyroid disease diagnosis with machine learning and counterfactual explainable AI," IEEE Access, vol. 14, pp. 80346-80370, 2026, doi: 10.1109/ACCESS.2026.3663497.

[2] A. Salekin and J. Stankovic, "Detection of chronic kidney disease and selecting important predictive attributes," in Proc. IEEE Int. Conf. Healthcare Informat. (ICHI), 2016, pp. 262-270, doi: 10.1109/ICHI.2016.36.

[3] M. A. Islam, M. Z. H. Majumder, and M. A. Hussein, "Chronic kidney disease prediction based on machine learning algorithms," J. Pathol. Inform., vol. 14, Art. no. 100189, 2023, doi: 10.1016/j.jpi.2023.100189.

[4] A. Sun and F. Sun, "Prediction of diabetes using supervised classification," J. Emerg. Investigators, Mar. 2024, doi: 10.59720/23-062.

[5] I. Straw and H. Wu, "Investigating for bias in healthcare algorithms: A sex-stratified analysis of supervised machine learning models in liver disease prediction," BMJ Health Care Inform., vol. 29, no. 1, Art. no. e100457, 2022, doi: 10.1136/bmjhci-2021-100457.

[6] R. Zannat, A. Al Shafi, and A. Muntakim, "Bridging the gap in Bangla healthcare: Machine learning based disease prediction using a symptoms-disease dataset," in Proc. Int. Conf. Elect., Comput. Commun. Eng. (ECCE), Chattogram, Bangladesh, Feb. 2025, doi: 10.1109/ECCE64574.2025.11012950 (arXiv:2601.12068).

[7] J. Al Nahian, A. K. M. Masum, S. Abujar, and M. J. Mia, "Common human diseases prediction using machine learning based on survey data," arXiv:2209.10750, Sep. 2022 (accepted in Bull. Elect. Eng. Informat.).

[8] V. Kamakshamma, G. M. Sumanth, K. R. G. N. K. Reddy, B. Pallavi, and C. S. Varshini, "Multi-disease prediction system using machine learning," in Proc. 1st Int. Conf. Res. Develop. Inf., Commun., Comput. Technol. (ICRDICCT), vol. 2. SCITEPRESS, 2025, pp. 116-127, doi: 10.5220/0013878600004919.

[9] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[10] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in Proc. Annu. Symp. Comput. Appl. Med. Care, 1988, pp. 261-265.

[11] S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," BMC Bioinformatics, vol. 7, Art. no. 91, 2006, doi: 10.1186/1471-2105-7-91.

[12] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Proc. Adv. Neural Inf. Process. Syst. (NIPS), vol. 30, 2017, arXiv:1705.07874.

[13] S. Wachter, B. Mittelstadt, and C. Russell, "Counterfactual explanations without opening the black box: Automated decisions and the GDPR," arXiv:1711.00399, 2017 (Harvard J. Law Technol., 2018).

[14] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artif. Intell. Res., vol. 16, pp. 321-357, 2002, doi: 10.1613/jair.953.

[15] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart Disease," UCI Machine Learning Repository, 1989 (303 instances, 13 features), doi: 10.24432/C52P4X.

[16] B. Ramana and N. Venkateswarlu, "ILPD (Indian Liver Patient Dataset)," UCI Machine Learning Repository, 2012 (583 instances, 10 features), doi: 10.24432/C5D02C.

[17] L. Rubini, P. Soundarapandian, and P. Eswaran, "Chronic Kidney Disease," UCI Machine Learning Repository, 2015 (400 instances, 24 features), doi: 10.24432/C5G020.

[18] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.

[19] C. Cortes and V. Vapnik, "Support-vector networks," Mach. Learn., vol. 20, no. 3, pp. 273-297, 1995, doi: 10.1007/BF00994018.